A8 Incremental processing

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A8 Incremental processing 3
A8-L1 Process only the waves you have not processed, correctly 3
A8-L2 Auto Loader for file ingestion into bronze 7
A8-L3 Change data feed, so downstream layers read only what changed 12
Track A best-practice checklist 17

Appendix: solutions and answer keys 18

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A8 Incremental processing

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Auto Loader, change data feed and processing only new waves.

A8-L1 Process only the waves you have not processed, correctly

Concept

Incremental loading means each run handles only new work: waves in bronze that silver does not have yet. It saves compute, and it keeps reruns cheap. It is also easy to get subtly wrong.

The pattern:

```
1 def unprocessed_waves(spark, bronze, silver, tracker_id) -> list[str]:
2     source = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
3     done = spark.table(silver).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
4     return sorted(r.wave_id for r in source.join(done, "wave_id",
    "left_anti").collect())
```

Four rules:

1. **Filter both sides the same way.** Your reference `get_unprocessed_waves` filters bronze by tracker but reads silver for every tracker. If tracker B already has wave `W14` in silver, tracker A's `W14` is treated as done and never loads. Scope both sides with the same keys.
2. **Use a left anti join** (`"left_anti"`): rows on the left with no match on the right. It says exactly what you mean, and it works for multi-column keys.
3. **Handle the first run.** When silver does not exist yet, every bronze wave is new. Check with `spark.catalog.tableExists(silver)`.
4. **Keep each wave's load idempotent.** Detection can be wrong after a partial failure, for example if silver has half a wave. Because the silver MERGE is idempotent (A3-L1), reprocessing a wave is always safe. Design for "process again", not "process exactly once".

Deciding by "waves not in silver" misses redeliveries of a wave silver already has. Either compare the latest `ingestion_timestamp` per wave in bronze with the one recorded when silver loaded it, or read bronze incrementally with change data feed (A8-L3) or Auto Loader (A8-L2).

Why it matters

Two trackers share one silver table. Tracker B loads wave `W14` on Monday. Tracker A's `W14` arrives on Tuesday. The reference `get_unprocessed_waves` compares tracker A's bronze waves against silver waves for all trackers, finds `W14` already present, and skips it. Tracker A's dashboard shows no `W14`, and no job fails. The gap is found nine days later, by a client.

Filtering silver by `tracker_id` too makes the comparison correct, and a reconciliation of bronze keys against silver rows (A6-L3) would have flagged it on Tuesday.

Worked example: weak code

```
1  def get_unprocessed_waves(bronze_table, silver_table, tracker_id):
2      bronze_waves = (
3          spark.table(bronze_table)
4          .filter(col("tracker_id") == tracker_id)
5          .select("wave_id").distinct()
6      )
7      silver_waves = spark.table(silver_table).select("wave_id").distinct()
8      return [row.wave_id for row in
       bronze_waves.subtract(silver_waves).collect()]
```

1. **Line 1.** Uses a global spark and has no type hints.
2. **Line 7.** No tracker filter: another tracker's wave counts as done (reference defect 10).
3. **Line 8.** Unsorted output, and it fails if silver does not exist yet.

Worked example: strong code

```
1  def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
       tracker_id: str) -> list[str]:
2      """Wave ids in bronze for this tracker that silver does not hold for
       this tracker."""
3      source = (
4          spark.table(bronze)
5          .filter(F.col("tracker_id") == tracker_id)
6          .select("wave_id")
7          .distinct()
8      )
9      if not spark.catalog.tableExists(silver):
10         return sorted(r.wave_id for r in source.collect())
11     done = (
12         spark.table(silver)
13         .filter(F.col("tracker_id") == tracker_id)
14         .select("wave_id")
15         .distinct()
16     )
```

```
❸     return sorted(r.wave_id for r in source.join(done, "wave_id",
    "left_anti").collect())
```

1. **Line 9.** First run: silver does not exist, so every wave is new.
2. **Line 12.** Silver is read with the same tracker filter as bronze.
3. **Line 17.** Left anti join: waves with no match in silver. Sorted, so waves load in order and logs are predictable.

Same filters on both sides, an explicit anti join, a first-run branch, and an idempotent load behind it.

Recap card

- **Rule for incremental wave detection** Filter bronze and silver by the same keys, such as tracker_id, then left-anti-join.
- **Join type for 'in A but not in B'** left_anti.
- **First run, silver missing** spark.catalog.tableExists(silver) is False: every bronze wave is new.
- **What does wave-ID detection miss?** Redeliveries of waves silver already holds. Use ingestion timestamps, change data feed or Auto Loader.

Practice

Exercise 1. Multiple choice (level 1)

Bronze for tracker A holds waves W13 and W14. Silver holds W13 for tracker A and W14 for tracker B. The reference function compares A's bronze waves with all of silver's waves. Which waves does it return?

- a) W14
- b) None
- c) W13 and W14
- d) W13

Why?

- r1) Tracker B's W14 in silver makes tracker A's W14 look done.
- r2) subtract removes all waves when silver is not empty.

Exercise 2. Fill the gap (level 2)

Keep only bronze waves that silver does not have, and handle a missing silver table.

```
1 if not spark.catalog._____(1)(silver):
2     return sorted(r.wave_id for r in source.collect())
```

```
3 new = source.join(done, "wave_id", "_____(2)")
```

Exercise 3. Spot the bug (level 3)

Tracker A's wave W14 never loads when tracker B already has W14. Tap the line, then fix the function.

```
1 def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
2
    """Waves in bronze for this tracker that silver does not hold for this
    tracker."""
3     source = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
4     done = spark.table(silver).select("wave_id").distinct()
5     return sorted(r.wave_id for r in source.join(done, "wave_id",
    "left_anti").collect())
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `unprocessed_waves(spark, bronze, silver, tracker_id)`. Return the sorted wave IDs present in bronze for the tracker and absent from silver for the same tracker. When silver does not exist, return every bronze wave for the tracker. Use a left anti join.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Wave ids in bronze for this tracker that silver does not hold for this
    tracker.
2 def unprocessed_waves(
3     spark: SparkSession,
4     bronze: str,
5     silver: str,
6     tracker_id: str,
7 ) -> list[str]: ...
```

Starter code:

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3
4
5 def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
6     ...
```

Exercise 5. Multiple choice (level 5)

Silver already holds wave W14. The supplier redelivers W14 with corrected weights. Your job processes "waves in bronze that are not in silver". What happens to the redelivery?

- a) It is processed, because bronze has a new ingestion_id.
- b) It is skipped, because W14 is already in silver.
- c) The job fails with a duplicate key error.
- d) It is processed twice.

Why?

- r1) The detection compares wave IDs only, and W14 exists on both sides.
- r2) Bronze rejects redeliveries.

Sources

- left_anti joins and spark.catalog.tableExists work as described on PySpark 4.0. (verified: spark_tests for A8-L1-E3 and E4 run on PySpark 4.0.4 with Delta.)

A8-L2 Auto Loader for file ingestion into bronze

Concept

Suppliers drop files into a landing folder. You want each file loaded into bronze exactly once, whenever it arrives, without keeping your own list of processed files. **Auto Loader** does this on Databricks. It is a Structured Streaming source called `cloudFiles`.

```
(
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", schema_path)
    .option("header", "true")
    .load(landing_path)
    .select("*", F.col("_metadata.file_path").alias("source_file"))
    .writeStream
    .option("checkpointLocation", checkpoint_path)
    .trigger(availableNow=True)
    .toTable("research_prod.research_bronze.nat_raw")
)
```

What each piece does:

- **Checkpoint location.** Auto Loader records every discovered file in a key-value store (RocksDB) under the checkpoint. After a failure it resumes

where it stopped, and each file path is processed once. One checkpoint per stream, never shared, never deleted casually: deleting it reloads every file.

- **Schema location.** Auto Loader infers the schema once, stores it there, and tracks changes. Without a schema you provide, the default mode is `addNewColumns`: when a new column appears, the stream stops with `UnknownFieldException` after recording the new schema, and the next start includes the column. Run it from a job that retries, so the restart is automatic.
- **Rescued data.** When Auto Loader infers the schema, it adds a `_rescued_data` column that captures values that do not fit, such as a type mismatch. Count non-null `_rescued_data` in your contract checks.
- `trigger(availableNow=True)` processes every file that has arrived, then stops. Schedule it as a batch job: incremental logic, batch cost.
- `_metadata.file_path` gives each row its source file, for the bronze audit columns.

Keep checkpoints and schemas in a Unity Catalog volume, such as `/Volumes/research_prod/research_bronze/checkpoints/nat_raw`.

Why it matters

A bronze notebook lists the landing folder, compares file names with a hand-maintained "processed.txt", and loads the rest. A timeout kills the notebook after it has loaded a file but before it has updated the list. The next run loads the same file again. Bronze now holds the delivery twice with two ingestion IDs, and silver's newest-version logic cannot tell them apart, because both carry the same `delivery_version`.

Auto Loader's checkpoint records files and commits the Delta write together, so a restart after a failure neither skips nor repeats a file.

Worked example: weak code

```
1  import os
2
1  done = set(open("/dbfs/tmp/processed.txt").read().split())
4  for name in os.listdir("/dbfs/landing/nat"):
5      if name not in done:
6          df = spark.read.option("header", True).csv(f"/landing/nat/{name}")
2          df.write.mode("append").saveAsTable("research_bronze.nat_raw")
3          open("/dbfs/tmp/processed.txt", "a").write(name + "\n")
```

1. **Line 3.** Hand-built state in a temp file: easy to lose, and not linked to the table.

2. **Line 7.** The write and the list update are separate steps. A crash between them duplicates a file.
3. **Line 8.** State recorded after the write, with no transaction.

Worked example: strong code

```
1  def start_bronze_ingest(spark: SparkSession, landing: str, table: str,
       checkpoint: str, schema_path: str) -> StreamingQuery:
2      """Load every new file in landing into bronze once, then stop."""
3      raw = (
1          spark.readStream.format("cloudFiles")
5          .option("cloudFiles.format", "csv")
2          .option("cloudFiles.schemaLocation", schema_path)
7          .option("header", "true")
8          .load(landing)
9      )
10     audited = raw.select(
11         "*",
3          F.col("_metadata.file_path").alias("source_file"),
13         F.current_timestamp().alias("ingestion_timestamp"),
14     )
15     return (
16         audited.writeStream.option("checkpointLocation", checkpoint)
4          .trigger(availableNow=True)
18         .toTable(table)
19     )
```

1. **Line 4.** Auto Loader discovers new files and tracks them in the checkpoint.
2. **Line 6.** Where the inferred schema and its changes are stored.
3. **Line 12.** Source file for every row, for the bronze audit columns.
4. **Line 17.** Process everything available, then stop: scheduled like a batch job.

Auto Loader replaces a fragile processed-files list with a checkpoint that commits together with the Delta write.

Recap card

- **What does Auto Loader's checkpoint store?** Discovered files (in RocksDB) and stream progress, so each file is processed exactly once across restarts.
- **Default schema evolution mode without a supplied schema** addNewColumns: stop with UnknownFieldException, record the new schema, restart.
- **Incremental at batch cost** .trigger(availableNow=True) on a scheduled job.
- **Checkpoint rule** One per stream, in durable storage such as a UC volume. Never shared, never deleted casually.

Practice

Exercise 1. Multiple choice (level 1)

What lets Auto Loader process each landing file exactly once across restarts?

- a) The schema location
- b) The checkpoint location
- c) The `_rescued_data` column
- d) The file names in the target table

Why?

- r1) Auto Loader records discovered files in a key-value store in the checkpoint and resumes from it.
- r2) The checkpoint deletes files after loading them.

Exercise 2. Fill the gap (level 2)

Complete the Auto Loader read and a write that processes all available files, then stops.

```
raw = (
    spark.readStream.format("_____(1)")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles._____(2)", schema_path)
    .load(landing)
)
raw.writeStream.option("checkpointLocation",
   checkpoint).trigger(_____(3)=True).toTable(table)
```

Exercise 3. Multiple choice (level 3)

An Auto Loader stream runs without a user-supplied schema. A new delivery adds a column `region`. What happens?

- a) The column is silently dropped.
- b) The stream stops with UnknownFieldException after recording the new schema. The next start includes region.
- c) The stream continues and region lands in _rescued_data, forever.
- d) The whole bronze table is rebuilt.

Why?

- r1) The default mode without a schema is addNewColumns, which evolves by failing once and restarting.
- r2) Auto Loader never changes a schema automatically.

Exercise 4. Spot the bug (level 4)

Two Auto Loader streams load different landing folders into two bronze tables. After a week, each table is missing files the other received. Tap the line, then fix the function.

```
1 CHECKPOINT = "/Volumes/research_prod/research_bronze/checkpoints/bronze"
2
3
4 def start_ingest(spark: SparkSession, landing: str, table: str,
    schema_path: str) -> StreamingQuery:
5     """Load new files from landing into table."""
6     raw =
    spark.readStream.format("cloudFiles").option("cloudFiles.format",
    "csv").option("cloudFiles.schemaLocation", schema_path).load(landing)
7     return raw.writeStream.option("checkpointLocation",
    CHECKPOINT).trigger(availableNow=True).toTable(table)
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Design the Auto Loader job for tracker nat. Choose each setting.

Options: trigger(availableNow=True), scheduled, A continuously running stream.

1. Files arrive a few times a day, and cost matters more than latency
2. Checkpoint location
3. What to do with _rescued_data

Sources

- Auto Loader tracks files in RocksDB in the checkpoint for exactly-once processing; _metadata.file_path is available; checkpoints can live in Volumes paths. (verified: https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/ (checked 2026-09-25))
- addNewColumns is the default schema evolution mode when no schema is provided; the stream stops with UnknownFieldException and picks up new columns on restart; _rescued_data is added when the schema is inferred. (verified: https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema (checked 2026-09-25))
- DataStreamWriter.trigger accepts availableNow=True and toTable exists in PySpark 4.0. (verified: Checked on PySpark 4.0.4 in this repository.)

A8-L3 Change data feed, so downstream layers read only what changed

Concept

When silver changes, gold should reprocess only the rows that changed, not the whole table. **Change data feed** (CDF) makes a Delta table record row-level changes for every write.

Turn it on per table:

```sql
ALTER TABLE research_prod.research_silver.nat_respondents
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

Read the changes since a version:

```python
changes = (
    spark.read.option("readChangeFeed", "true")
    .option("startingVersion", last_version + 1)
    .table("research_prod.research_silver.nat_respondents")
)
```

Each row carries three extra columns:

- `_change_type`: `insert`, `update_preimage` (the row before an update), `update_postimage` (the row after), or `delete`.
- `_commit_version` and `_commit_timestamp`: which commit made the change.

Two rules make CDF consumers correct:

1. **Drop `update_preimage` rows** before you use the changes. An update produces two rows, old and new. Keep the old one and every updated respondent is counted twice.
2. **Keep one row per key**: the latest by `_commit_version`. A respondent updated in three commits since your last run appears three times.

Track progress. Either store the last processed version in a state table, or read the feed as a stream (`readStream` with `readChangeFeed`) and let the checkpoint remember it. Change data only exists from the version where you enabled it. Asking for an earlier version fails with `DELTA_MISSING_CHANGE_DATA`. VACUUM also removes old change files after the retention window.

For gold, collect the waves and subgroups the changes touch, then recompute just those aggregates. Aggregates cannot be patched row by row.

Why it matters

A gold job rebuilds weighted metrics for every wave every night: 180 waves, 40 minutes. Most nights, only one wave changed. When a late correction

arrives mid-afternoon, the team cannot refresh gold before the client call, because a full rebuild takes too long.

With CDF on silver, the job reads the day's changes, finds the one wave they touch, and recomputes it in under two minutes. The same job also stops double counting, because it drops `update_preimage` rows before aggregating.

Worked example: weak code

```
1 def changed_rows(spark, table, since):
2     changes = (
3         spark.read.option("readChangeFeed", "true")
2         .option("startingVersion", since)
5         .table(table)
6     )
3     return changes.filter(F.col("_change_type") != "delete")
```

1. **Line 1.** A respondent updated in several commits still appears once per commit.
2. **Line 4.** If since is the last processed version, that version is read again.
3. **Line 7.** Keeps update_preimage rows: every updated respondent appears twice, once with old values.

Worked example: strong code

```
1  CURRENT_STATE = ["insert", "update_postimage"]
2
3
4  def read_changes(spark: SparkSession, table: str, last_processed_version:
   int) -> DataFrame:
5      """Row changes committed after last_processed_version."""
6      return (
7          spark.read.option("readChangeFeed", "true")
1          .option("startingVersion", last_processed_version + 1)
9          .table(table)
10     )
11
12
13 def latest_state(changes: DataFrame, keys: list[str]) -> DataFrame:
14     """One row per key: its newest change, excluding pre-images."""
2      window =
   Window.partitionBy(*keys).orderBy(F.col("_commit_version").desc())
16     return (
3          changes.filter(F.col("_change_type") != "update_preimage")
18         .withColumn("_rank", F.row_number().over(window))
19         .filter(F.col("_rank") == 1)
20         .drop("_rank")
21     )
22
23
4  def touched_waves(changes: DataFrame) -> list[str]:
25     """Waves with any change: the only gold waves to recompute."""
```

```
26      return sorted(r.wave_id for r in
        changes.select("wave_id").distinct().collect())
```

1. **Line 8.** Start after the last processed version, so nothing is read twice.
2. **Line 15.** Several changes to one key since the last run: keep the newest. Deletes stay, so the consumer can remove the key.
3. **Line 17.** Pre-images describe old state. Drop them.
4. **Line 24.** Gold recomputes whole aggregates, but only for the waves that changed.

Read after the last version, drop pre-images, keep the newest change per key, and recompute only what the changes touch.

Recap card

- **Turn on change data feed** ALTER TABLE t SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
- **The four _change_type values** insert, update_preimage (old), update_postimage (new), delete.
- **Two rules for consuming CDF** Drop update_preimage. Keep the newest change per key by _commit_version.
- **Where does the next run start?** startingVersion = last processed version + 1 (it is inclusive), or a streaming checkpoint.

Practice

Exercise 1. Multiple choice (level 1)

An UPDATE changes one respondent's weight. Which change feed row holds the new weight?

- a) insert
- b) update_preimage
- c) update_postimage
- d) delete

Why?

- r1) An update records the row before (pre-image) and after (post-image). The post-image holds the new values.
- r2) Updates are recorded as a delete followed by an insert.

Exercise 2. Predict the output (level 2)

CDF is enabled when the table is created. How many rows does the change read return?

What you need to know.

In SQL, `table_changes('table_name', start_version)` returns the same rows as reading the change feed with `readChangeFeed`: one row per change from `start_version` onwards, with `_change_type`, `_commit_version` and `_commit_timestamp`.

`TBLPROPERTIES (...)` inside `CREATE TABLE` sets table properties at creation, as `ALTER TABLE ... SET TBLPROPERTIES` does later.

```sql
CREATE TABLE c1 (id INT, v STRING) USING DELTA
  TBLPROPERTIES (delta.enableChangeDataFeed = true);   -- version 0
INSERT INTO c1 VALUES (1, 'a'), (2, 'b');              -- version 1
UPDATE c1 SET v = 'z' WHERE id = 1;                    -- version 2
DELETE FROM c1 WHERE id = 2;                           -- version 3

SELECT COUNT(*) FROM table_changes('c1', 2);
```

Exercise 3. Fill the gap (level 3)

Read silver's changes committed after the last processed version.

```python
changes = (
    spark.read.option("_____(1)", "true")
    .option("startingVersion", _____(2))
    .table(silver)
)
```

Exercise 4. Write the function (level 4)

Write `latest_state(changes, keys)`. From a change feed DataFrame, drop `update_preimage` rows, then keep one row per key: the one with the highest `_commit_version`. Use a window with `row_number`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# One row per key: its newest change, excluding update pre-images.
def latest_state(
    changes: DataFrame,
    keys: list[str],
) -> DataFrame: ...
```

Starter code:

```python
from pyspark.sql import DataFrame, Window
from pyspark.sql import functions as F


def latest_state(changes: DataFrame, keys: list[str]) -> DataFrame:
    ...
```

Exercise 5. Spot the bug (level 5)

After enabling CDF, updated respondents appear twice in the gold input, once with old values. Tap the line, then fix the function.

What you need to know.

`F.col(name).isin([...])` is true when the column's value equals any item in the list. It keeps several allowed values in one condition:

```
1 df.filter(F.col("region").isin(["North", "South"]))
```

```
1 def current_rows(spark: SparkSession, table: str, last_processed_version:
    int) -> DataFrame:
2     """Inserted and updated rows since the last run, with their new
    values."""
3     changes = spark.read.option("readChangeFeed",
    "true").option("startingVersion", last_processed_version +
    1).table(table)
4     return changes.filter(F.col("_change_type") != "delete")
```

Write down the line number of the bug. Then write the fix.

Practical: Read a change data feed and keep the latest state

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It enables change data feed on a small silver table, applies an insert, an update and a delete, then reads the changes and reduces them to the latest state per respondent.

Download the notebook `A8-L3.ipynb` from the app.

Sources

- CDF read semantics, the four change types, table_changes, and DELTA_MISSING_CHANGE_DATA for versions before CDF was enabled. (verified: Run on delta-spark 4.0.1 in this repository.)
- CDF on Databricks: delta.enableChangeDataFeed, readChangeFeed, startingVersion, table_changes. (verified: https://docs.databricks.com/aws/en/tables/features/change-data-feed (checked 2026-09-25))

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A8-L1 Process only the waves you have not processed, correctly

A8-L1 exercise 1

Hints. (1) Which silver waves does the function compare against? (2) Is W14 in silver, for any tracker? (3) Both of A's waves appear in silver.

Answer. b) None Because: r1) Tracker B's W14 in silver makes tracker A's W14 look done.

Without a tracker filter on silver, any tracker's wave counts as done. A's W14 never loads, and nothing fails.

Common wrong answers.

- `a`: That is the correct answer to the question the function should ask. Because silver is not filtered by tracker, B's W14 hides A's.
- `c`: W13 is in silver for tracker A, so it is correctly excluded. W14 is wrongly excluded too.
- `d`: W13 is already loaded for tracker A.
- `b|r2`: Right outcome, wrong reason. subtract removes only matching values. Here every value matches, because silver mixes trackers.

A8-L1 exercise 2

Hints. (1) The Catalog method that checks whether a table exists. (2) The join type that keeps left rows without a match. (3) `tableExists` and `left_anti`.

Answer.

```
if not spark.catalog.tableExists(silver):
    return sorted(r.wave_id for r in source.collect())
new = source.join(done, "wave_id", "left_anti")
```

`spark.catalog.tableExists` covers the first run. A `left_anti` join keeps bronze waves with no matching silver wave.

Common wrong answers.

- g2 = `left`: A left join keeps every bronze wave, matched or not. You want the ones with no match: left_anti.
- g2 = `left_semi`: left_semi keeps the waves that do match: the ones already done. You want the opposite.
- g1 = `exists`: The Catalog method is `tableExists`.

A8-L1 exercise 3

Hints. (1) Compare lines 3 and 4. (2) Which side lacks the tracker filter? (3) Add `.filter(F.col("tracker_id") == tracker_id)` to silver.

Answer. The bug is on line 4.

Fixed version:

```
1 def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
2
    """Waves in bronze for this tracker that silver does not hold for this
    tracker."""
3     source = spark.table(bronze).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
4     done = spark.table(silver).filter(F.col("tracker_id") ==
    tracker_id).select("wave_id").distinct()
5     return sorted(r.wave_id for r in source.join(done, "wave_id",
    "left_anti").collect())
```

Silver is read for all trackers, so another tracker's wave hides this tracker's wave. Filter both sides by the same keys.

Common wrong answers.

- Line 3: Bronze is filtered by tracker, which is right. Compare it with how silver is read.
- Line 5: The left anti join is correct once both sides cover the same tracker.

A8-L1 exercise 4

Hints. (1) Build the bronze waves for the tracker first. (2) Return them all if silver does not exist. (3) Otherwise filter silver the same way and left-anti-join.

Answer.

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3
4
5 def unprocessed_waves(spark: SparkSession, bronze: str, silver: str,
    tracker_id: str) -> list[str]:
```

```
    """Wave ids in bronze for this tracker that silver does not hold for this tracker.

    Args:
        spark: Active session.
        bronze: Bronze table name.
        silver: Silver table name. May not exist yet.
        tracker_id: Tracker to scope both sides to.

    Returns:
        Sorted wave ids to process.
    """
    source = spark.table(bronze).filter(F.col("tracker_id") == tracker_id).select("wave_id").distinct()
    if not spark.catalog.tableExists(silver):
        return sorted(r.wave_id for r in source.collect())
    done = spark.table(silver).filter(F.col("tracker_id") == tracker_id).select("wave_id").distinct()
    return sorted(r.wave_id for r in source.join(done, "wave_id", "left_anti").collect())
```

Scoping both sides by tracker, handling the missing table and anti-joining gives the exact set of new waves. Sorting keeps runs deterministic.

A8-L1 exercise 5

Hints. (1) What does the detection compare? (2) Is W14 on both sides? (3) Redeliveries need a signal beyond the wave ID.

Answer. b) It is skipped, because W14 is already in silver. Because: r1) The detection compares wave IDs only, and W14 exists on both sides.

Wave-ID detection cannot see redeliveries. Compare the latest ingestion timestamp per wave, or read bronze incrementally with change data feed or Auto Loader.

Common wrong answers.

- `a`: The detection never looks at ingestion_id. It compares wave IDs, and W14 is in silver.
- `c`: Nothing runs for W14, so nothing can fail.
- `d`: W14 is not selected at all.
- `b|r2`: Right outcome, wrong reason. Bronze appends every redelivery. The detection is what skips it.

A8-L2 Auto Loader for file ingestion into bronze

A8-L2 exercise 1

Hints. (1) Where does a stream keep its progress? (2) It is a separate path, given in the writeStream options. (3) checkpointLocation.

Answer. b) The checkpoint location Because: r1) Auto Loader records discovered files in a key-value store in the checkpoint and resumes from it.

The checkpoint holds a RocksDB store of discovered files and the stream's progress. On restart, Auto Loader continues from it, so files are neither skipped nor repeated.

Common wrong answers.

- `a`: The schema location stores the inferred schema, not the list of processed files.
- `c`: _rescued_data holds values that did not fit the schema. It says nothing about which files were loaded.
- `d`: Auto Loader never reads the target table to decide what to load. The checkpoint does that.
- `b|r2`: Right answer, wrong reason. Auto Loader leaves source files in place. It records them in the checkpoint.

A8-L2 exercise 2

Hints. (1) The source format has the same prefix as its options. (2) Where the inferred schema is stored. (3) The trigger that processes everything available, then stops.

Answer.

```
raw = (
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", schema_path)
    .load(landing)
)
raw.writeStream.option("checkpointLocation",
   checkpoint).trigger(availableNow=True).toTable(table)
```

`format("cloudFiles")` selects Auto Loader. `cloudFiles.schemaLocation` stores the schema. `trigger(availableNow=True)` runs the stream as a batch.

Common wrong answers.

- g1 = `autoloader`: The source format is named `cloudFiles`.

- g3 = `once`: `once=True` processes everything in a single micro-batch. `availableNow=True` also stops when done, but can split a large backlog into several batches, which Databricks recommends for Auto Loader.
- g2 = `checkpointLocation`: The checkpoint goes on the writer. The reader option for the schema store is `cloudFiles.schemaLocation`.

A8-L2 exercise 3

Hints. (1) Which mode applies when you do not provide a schema? (2) How does that mode pick up a new column? (3) Fail once, restart with the new schema.

Answer. b) The stream stops with UnknownFieldException after recording the new schema. The next start includes region. Because: r1) The default mode without a schema is addNewColumns, which evolves by failing once and restarting.

Without a schema, the default is `addNewColumns`: the stream stops with `UnknownFieldException`, the schema location gains the column, and a restart continues. Jobs with retries make that restart automatic.

Common wrong answers.

- `a`: Nothing is dropped silently. In addNewColumns mode the stream stops so the new column is picked up.
- `c`: That describes the `rescue` mode. The default without a schema is `addNewColumns`.
- `d`: Auto Loader never rebuilds the target. It evolves the schema for new files.
- `b|r2`: Right outcome, wrong reason. It does change the schema: it records the evolved schema, then needs a restart.

A8-L2 exercise 4

Hints. (1) How many streams share the same checkpoint? (2) A checkpoint records one stream's progress. (3) Give each stream its own checkpoint path.

Answer. The bug is on line 1.

Fixed version:

```
1 def start_ingest(spark: SparkSession, landing: str, table: str,
    schema_path: str, checkpoint: str) -> StreamingQuery:
2     """Load new files from landing into table. checkpoint must be unique
    to this stream."""
3     raw =
    spark.readStream.format("cloudFiles").option("cloudFiles.format",
    "csv").option("cloudFiles.schemaLocation", schema_path).load(landing)
```

```
4     return raw.writeStream.option("checkpointLocation",
      checkpoint).trigger(availableNow=True).toTable(table)
```

Both streams write their progress to the same checkpoint, so each reads the other's state and skips files. One checkpoint per stream, passed in from config.

Common wrong answers.

- Line 6: The read is fine, with its own schema location. The problem is on the write side.

A8-L2 exercise 5

Hints. (1) Latency or cost? (2) The checkpoint must be durable and private to the stream. (3) Bronze keeps evidence.

Answer.

- availableNow on a schedule: incremental processing at batch cost.
- A dedicated checkpoint per stream in a Unity Catalog volume, so it survives and is governed.
- Keep _rescued_data in bronze and count it in the contract checks: it is evidence of schema drift.

Scheduled availableNow, a durable per-stream checkpoint in a volume, and rescued data counted as a quality signal.

Common wrong answers.

- trigger = `continuous`: A continuous stream keeps compute running all day for a few files. Scheduled availableNow costs far less.
- checkpoint = `tmp`: Temporary storage can be cleared. Losing the checkpoint reloads every file.
- checkpoint = `shared`: Streams must not share a checkpoint. They would skip each other's files.
- rescued = `drop`: Dropping _rescued_data in bronze destroys evidence. Bronze keeps everything.

A8-L3 Change data feed, so downstream layers read only what changed

A8-L3 exercise 1

Hints. (1) An update records two versions of the row. (2) Which one comes after the change? (3) post = after.

Answer. c) update_postimage Because: r1) An update records the row before (pre-image) and after (post-image). The post-image holds the new values.

An update writes `update_preimage` (old values) and `update_postimage` (new values). Use the post-image, and drop the pre-image.

Common wrong answers.

- `a`: insert marks a new row. An update of an existing row produces a pre-image and a post-image.
- `b`: The pre-image is the row before the update, with the old weight.
- `d`: delete marks a removed row.
- `c|r2`: Right row type, wrong reason. Updates appear as update_preimage and update_postimage, not delete plus insert.

A8-L3 exercise 2

Hints. (1) Which versions does table_changes('c1', 2) read? (2) How many rows does one updated row produce? (3) Two for the update, one for the delete.

Answer.

```
3
```

From version 2: `update_preimage` and `update_postimage` for id 1, and `delete` for id 2. Three rows. This was run on delta-spark 4.0.1.

Common wrong answers.

- `2`: The update produces two rows, pre-image and post-image, plus one delete row: three.
- `5`: Version 1's inserts come before the starting version 2, so they are not included.

A8-L3 exercise 3

Hints. (1) The read option names what you are reading. (2) startingVersion is inclusive. (3) `readChangeFeed` and `last_processed_version + 1`.

Answer.

```
changes = (
    spark.read.option("readChangeFeed", "true")
    .option("startingVersion", last_processed_version + 1)
    .table(silver)
)
```

`readChangeFeed` switches the read to the change feed. Starting at `last_processed_version + 1` reads each change exactly once.

Common wrong answers.

- g2 = `last_processed_version`: startingVersion is inclusive. Starting at the last processed version reads its changes twice.
- g1 = `enableChangeDataFeed`: `delta.enableChangeDataFeed` is the table property. The read option is `readChangeFeed`.
- g2 = `0`: Version 0 rereads all history every run, and fails if CDF was enabled later.

A8-L3 exercise 4

Hints. (1) Filter out update_preimage first. (2) Window: partition by keys, order by _commit_version descending. (3) Keep row_number 1, drop the helper column.

Answer.

```python
from pyspark.sql import DataFrame, Window
from pyspark.sql import functions as F


def latest_state(changes: DataFrame, keys: list[str]) -> DataFrame:
    """One row per key: its newest change, excluding update pre-images.

    Args:
        changes: Rows read with readChangeFeed.
        keys: Columns identifying one row of the source table.

    Returns:
        The newest insert, update_postimage or delete per key.
    """
    window = Window.partitionBy(*keys).orderBy(F.col("_commit_version").desc())
    return (
        changes.filter(F.col("_change_type") != "update_preimage")
        .withColumn("_rank", F.row_number().over(window))
        .filter(F.col("_rank") == 1)
        .drop("_rank")
    )
```

Pre-images share a commit version with their post-images, so without the filter a pre-image can win the ranking and bring back old values. Ranking by version keeps the newest change per key.

A8-L3 exercise 5

Hints. (1) Which change types are not deletes? (2) One of them holds the old values. (3) Keep insert and update_postimage only.

Answer. The bug is on line 4.

Fixed version:

```
1 def current_rows(spark: SparkSession, table: str, last_processed_version: int) -> DataFrame:
2     """Inserted and updated rows since the last run, with their new values."""
3     changes = spark.read.option("readChangeFeed", "true").option("startingVersion", last_processed_version + 1).table(table)
4     return changes.filter(F.col("_change_type").isin(["insert", "update_postimage"]))
```

Excluding only deletes still keeps `update_preimage`, the old version of each updated row. Name the change types you want: `insert` and `update_postimage`.

Common wrong answers.

- Line 3: The read starts after the last processed version, which is correct.